Chapter 3: Selections

COMP 110
Worked examples, discussion, and quizzes

Learning goals

Write and trace boolean conditions
Choose if, if-else, or an else-if chain
Combine conditions and use short-circuit evaluation
Generate small random integers
Use classic and modern switch forms
Test decisions at their boundaries

A calculation may need a decision

Chapter 2 computed an area for any entered radius.
What should happen for a negative radius?

A selection chooses which statements run.
The choice depends on a true-or-false condition.

Discussion: Is zero a valid radius for our model?

A condition produces a boolean

```
int radius = 5;
boolean positive = radius > 0;
boolean negative = radius < 0;
System.out.println(positive);
System.out.println(negative);
System.out.println(1 > 2);
```

boolean values are true and false; they are not integers.

Java: BooleanBasics.java

Comparisons for radius = 5

Expression	Meaning and result
radius < 0 / radius <= 0	Less than / at most zero: false / false
radius > 0 / radius >= 0	Greater than / at least zero: true / true
radius == 0	Equal to zero: false
radius != 0	Different from zero: true

Use == for equality and = for assignment.

All six comparisons in Java

```
int radius = 5;
System.out.println(radius < 0);
System.out.println(radius <= 0);
System.out.println(radius > 0);
System.out.println(radius >= 0);
System.out.println(radius == 0);
System.out.println(radius != 0);
```

Predict the results again after changing radius to zero.

Java: RelationalOperators.java

Quiz 1: comparisons

Assume int value = 0.

1. value < 0
2. value <= 0
3. value == 0
4. value != 0
5. Why is if (value = 0) invalid?

Java: ComparisonQuiz.java

One-way if: optionally run a block

```
Scanner input = new Scanner(System.in);
System.out.print("Enter score: ");
int score = input.nextInt();
if (score >= 90) {
    System.out.println("High score");
}
System.out.println("Finished");
```

Finished is outside the block and runs on either path.

Java: OneWayIf.java

Two independent tests can both succeed

```
if (number % 5 == 0) {
    System.out.println("Hi Five");
}
if (number % 2 == 0) {
    System.out.println("Hi Even");
}
```

For 10, both messages print. These are two separate decisions.

Java: SimpleIfDemo.java

Tracing independent conditions

Input number	Messages printed
10	Hi Five, then Hi Even
15	Hi Five
8	Hi Even
7	No matching message

Every condition is evaluated in sequence.

Java: SimpleIfDemo.java

Two-way if-else: exactly one branch

```
if (radius >= 0) {
    double area = radius * radius * 3.14159;
    System.out.println("Area = " + area);
} else {
    System.out.println("Negative radius");
}
```

A nonnegative radius computes an area; a negative one prints an error.

Java: CircleAreaSelection.java

Quiz 2: follow the path

1. Which messages print for number 30?
2. Which messages print for number 9?
3. Can both branches of one if-else run?
4. Does a statement after the if-else run?
5. Which circle branch handles radius zero?

An else-if chain chooses one category

Example grade policy for scores from 0 to 100:

90 or above: A 80 to below 90: B
70 to below 80: C 60 to below 70: D
Below 60: F

Test the highest threshold first.

A grade classifier

```
if (score >= 90) {
    System.out.println("A");
} else if (score >= 80) {
    System.out.println("B");
} else if (score >= 70) {
    System.out.println("C");
} else if (score >= 60) {
    System.out.println("D");
} else {
    System.out.println("F");
}
```

After score is validated, only the first matching category prints.

Java: GradeClassifier.java

Tracing score = 70

Step	Decision or result
score >= 90	false; continue
score >= 80	false; continue
score >= 70	true; print C
Remaining branches	Skipped; leave the chain

The else-if chain produces one grade.

Java: GradeClassifier.java

A broad condition can hide a later one

```
int score = 95;
if (score >= 60) {
    System.out.println("D");
} else if (score >= 90) {
    System.out.println("A");
}
```

This intentional error prints D for a score of 95.

Java: GradeOrderPitfall.java

Independent if statements do not form a chain

```
int score = 95;
if (score >= 90) System.out.println("A");
if (score >= 80) System.out.println("B");
if (score >= 70) System.out.println("C");
if (score >= 60) System.out.println("D");
```

All four conditions are true for 95, so four letters print.

Java: IndependentGradesPitfall.java

Discussion: test around each threshold

For the grade classifier, choose tests near 80.
What should happen at 79.9, 80, and 80.1?

Also test 0, 100, an invalid low value,
and an invalid high value.

Why is testing only 95 insufficient?

Nested if: a second decision inside a branch

```
if (age >= 18) {
    if (hasTicket) {
        System.out.println("Admit");
    } else {
        System.out.println("Ticket required");
    }
} else {
    System.out.println("Age requirement not met");
}
```

The inner condition is reached only after the age test succeeds.

Java: NestedIf.java

Which if owns this else?

```
int i = 1, j = 2, k = 3;
if (i > j)
    if (i > k)
        System.out.println("A");
    else
        System.out.println("B");
System.out.println("Finished");
```

The else belongs to the nearest unmatched if.

Java: DanglingElse.java

Braces make the intended pairing explicit

```
int i = 1, j = 2, k = 3;
if (i > j) {
    if (i > k) {
        System.out.println("A");
    }
} else {
    System.out.println("B");
}
```

The outer condition is false, so this version prints B.

Java: BracedElse.java

A stray semicolon creates an empty statement

```
int score = 50;
if (score >= 90); // accidental empty statement
{
    System.out.println("High score");
}
System.out.println("Finished");
```

The following block runs independently and prints High score.

Java: SemicolonPitfall.java

An accidental boolean assignment can compile

```
boolean even = false;
if (even = true) {
    System.out.println("Branch taken");
}
System.out.println(even);
```

This assigns true to even, then uses that assigned value as the condition.

Java: BooleanAssignmentPitfall.java

Store a comparison result directly

```
int number = 8;
boolean even = number % 2 == 0;
if (even) {
    System.out.println("Even");
}
System.out.println(!even);
```

A comparison already returns the boolean value you need.

Java: BooleanIdiom.java

Quiz 3: diagnose the selection bug

1. What does if (score >= 90); control?
2. Does indentation choose the owner of else?
3. Can if (flag = true) compile?
4. What is wrong with testing score >= 60 before score >= 90 in a grade chain?

Math.random and a small integer range

```
double sample = Math.random();
int digit = (int) (sample * 10);
System.out.println("Sample: " + sample);
System.out.println("Digit: " + digit);
```

Math.random() produces a value at least 0.0 and below 1.0.

Java: RandomDigit.java

Cast after scaling the random fraction

```
double sample = 0.73;
int correct = (int) (sample * 10);
int wrong = (int) sample * 10;
int die = 1 + (int) (sample * 6);
System.out.println(correct);
System.out.println(wrong);
System.out.println(die);
```

For sample 0.73, the outputs are 7, 0, and 5.

Java: RandomRangeCast.java

A random addition question

```
int first = (int) (Math.random() * 10);
int second = (int) (Math.random() * 10);
System.out.println("What is " + first + " + " + second + "?");
int answer = input.nextInt();
if (answer == first + second) {
    System.out.println("Correct");
} else {
    System.out.println("Correct answer: " + (first + second));
}
```

The question changes; the same condition checks every answer.

Java: AdditionQuiz.java

Keep a subtraction question nonnegative

```
if (first < second) {
    int temporary = first;
    first = second;
    second = temporary;
}
System.out.println(first + " - " + second);
System.out.println(first - second);
```

Save the old first value before replacing it.

Java: SwapForSubtraction.java

An improved subtraction quiz

```
if (first < second) {
    int temporary = first;
    first = second;
    second = temporary;
}
System.out.println("What is " + first + " - " + second + "?");
int answer = input.nextInt();
if (answer == first - second) {
    System.out.println("Correct");
} else {
    System.out.println("Correct answer: " + (first - second));
}
```

The full file generates two digits before this snippet.

Java: SubtractionQuiz.java

Discussion: how can a random program be tested?

Check properties that every question must satisfy.

Both operands must be within 0 to 9.
Subtraction must put the larger operand first.
Correct answers must reach the correct branch.
Wrong answers must show the actual answer.

Gross pay: define the overtime rule

Sample rule for this exercise:
First 40 hours: ordinary hourly rate
Hours above 40: 1.5 times the rate

At $20/hour, 45 hours should pay $950.
Only the extra 5 hours receive the higher rate.

Gross pay with one overtime band

```
double pay;
if (hours <= 40) {
    pay = hours * rate;
} else {
    pay = 40 * rate + (hours - 40) * rate * 1.5;
}
System.out.printf(Locale.US, "Pay: $%.2f%n", pay);
```

Validate the inputs, then choose the regular or overtime calculation.

Java: GrossPay.java

A second rule: two overtime bands

Hours in this band	Rate for hours in that band
0 through 40	1.0 × hourly rate
Above 40 through 54	1.5 × hourly rate
Above 54	2.0 × hourly rate

Each multiplier applies only to the hours in its own band.

An else-if chain for tiered gross pay

```
double pay;
if (hours <= 40) {
    pay = hours * rate;
} else if (hours <= 54) {
    pay = 40 * rate + (hours - 40) * rate * 1.5;
} else {
    pay = 40 * rate + 14 * rate * 1.5
        + (hours - 54) * rate * 2;
}
System.out.printf(Locale.US, "Pay: $%.2f%n", pay);
```

The last branch includes the completed lower bands.

Java: TieredGrossPay.java

Quiz 4: pay boundaries

Use the two-band rule with hourly rate $20.

1. Pay for 40 hours?
2. Pay for 41 hours?
3. Pay for 54 hours?
4. Pay for 55 hours?
5. Why test both 54 and 55?

Java: TieredGrossPay.java

Logical operators combine conditions

Operator	Meaning for boolean operands
!p	Negate p.
p && q	True only when both are true.
p || q	True when at least one is true.
p ^ q	True when exactly one is true.

With boolean operands, ^ is exclusive OR, not exponentiation.

AND and OR truth table

p, q	p && q	p || q
false, false	false	false
false, true	false	true
true, false	false	true
true, true	true	true

One true input is enough for OR. Both are needed for AND.

NOT and exclusive OR

Input	Result
p = false / p = true	!p is true / false
p = false, q = false	p ^ q is false
Exactly one of p, q is true	p ^ q is true
p = true, q = true	p ^ q is false

Two false inputs do not make XOR true.

Checking the logical examples

```
int age = 24, weight = 140;
System.out.println(!(age > 18));
System.out.println(!(weight == 150));
System.out.println((age > 18) && (weight >= 140));
System.out.println((age > 14) || (weight >= 150));
System.out.println((age > 34) ^ (weight > 140));
```

Predict false, true, true, true, false.

Java: LogicalOperators.java

Divisibility: both, either, or exactly one

```
boolean byTwo = number % 2 == 0;
boolean byThree = number % 3 == 0;
System.out.println("Both: " + (byTwo && byThree));
System.out.println("Either: " + (byTwo || byThree));
System.out.println("Exactly one: " + (byTwo ^ byThree));
```

For 6, both is true and exactly one is false.

Java: TestBooleanOperators.java

A range needs two comparisons

```
boolean valid = Double.isFinite(score)
    && score >= 0 && score <= 100;
System.out.println("Valid: " + valid);
// 0 <= score <= 100 is invalid Java.
```

Use && for an inclusive range and reject nonfinite values when needed.

Java: RangeCheck.java

Short-circuit AND can protect a calculation

```
boolean safe = divisor != 0
    && number / divisor > 2;
System.out.println("Result: " + safe);
```

When divisor is zero, the right operand is not evaluated.

Java: ShortCircuitAnd.java

Trace which operands actually execute

```
int checks = 0;
boolean first = true || (++checks > 0);
System.out.println(first + " " + checks);
boolean second = false && (++checks > 0);
System.out.println(second + " " + checks);
boolean third = true ^ (++checks > 0);
System.out.println(third + " " + checks);
```

|| and && can skip the increment. Boolean ^ evaluates both sides.

Java: ShortCircuitTrace.java

Negating a combined condition

```
int age = 24, weight = 140;
boolean first = !(age > 18 && weight >= 140);
boolean second = age <= 18 || weight < 140;
boolean third = !(age > 18 || weight >= 140);
boolean fourth = age <= 18 && weight < 140;
System.out.println(first + " " + second);
System.out.println(third + " " + fourth);
```

Negate each comparison and exchange AND with OR.

Java: DeMorganDemo.java

Quiz 5: logical conditions

1. true || false
2. true ^ true
3. !(false && true)
4. How do you test age from 18 through 25?
5. Why place divisor != 0 before a division
   in an && expression?

Java: LogicalQuizCheck.java

Leap years: translate the whole rule

Gregorian arithmetic rule for this exercise:

Divisible by 4 and not divisible by 100,
or divisible by 400.

1900 is not a leap year. 2000 is a leap year.
Test the century exception, not only 2024.

Leap-year condition in Java

```
boolean leap = (year % 4 == 0 && year % 100 != 0)
    || year % 400 == 0;
System.out.println("Leap year: " + leap);
```

The parentheses show the two alternatives in the rule.

Java: LeapYear.java

Leap-year test cases

Year	Expected result and reason
2023	false: not divisible by 4
2024	true: divisible by 4, not by 100
1900	false: century, not divisible by 400
2000	true: divisible by 400

Each case targets a different part of the rule.

Java: LeapYear.java

Lottery exercise: decide the matching rule

Use a fixed draw of 37 while testing.
Accept a two-digit guess from 10 to 99.

Exact order match: 10,000 points
Both digits reversed: 3,000 points
One matching digit: 1,000 points
No matching digit: 0 points

Lottery matching: specific cases first

```
int prize;
if (guess == draw) {
    prize = 10000;
} else if (g1 == d2 && g2 == d1) {
    prize = 3000;
} else if (g1 == d1 || g1 == d2 || g2 == d1 || g2 == d2) {
    prize = 1000;
} else {
    prize = 0;
}
System.out.println("Points: " + prize);
```

The full file splits each number into tens and ones digits.

Java: LotteryDemo.java

Testing all lottery outcomes

Guess for draw 37	Expected points
37	10,000: exact order
73	3,000: reversed order
35 or 33	1,000: at least one shared digit
55	0: no shared digit

Also test 9 and 100 to check input rejection.

Java: LotteryDemo.java

Classic switch: select a labeled branch

```
switch (status) {
    case 0: System.out.println("Single"); break;
    case 1: System.out.println("Joint"); break;
    case 2: System.out.println("Separate"); break;
    case 3: System.out.println("Head of household"); break;
    default: System.out.println("Invalid status");
}
```

A matching case starts execution. break leaves the switch.

Java: ClassicSwitch.java

Rules for the classic switch forms here

Use int, char, String, or an enum selector.
Related small integral types are also supported.
Case constants must be compatible and distinct.
A classic switch can omit default.
Without break, execution may continue into later cases.

A colon-label switch can fall through

```
switch (day) {
    case 1: System.out.println("One");
    case 2: System.out.println("Two");
    case 3: System.out.println("Three"); break;
    default: System.out.println("Other");
}
```

For day 2, this prints Two and then Three.

Java: SwitchFallThrough.java

Tracing day = 2 without a break at case 2

Step	Execution
Find entry label	case 2
Print Two	Continue; no break
Reach case 3	Print Three
break	Leave the switch

A case label marks an entry point; it is not a new if test.

Java: SwitchFallThrough.java

Shared labels can be deliberate

```
switch (day) {
    case 1: case 2: case 3: case 4: case 5:
        System.out.println("Weekday"); break;
    case 0: case 6:
        System.out.println("Weekend"); break;
    default:
        System.out.println("Invalid day");
}
```

Several case labels can lead to the same statement group.

Java: SharedCaseLabels.java

Arrow switch rules do not fall through

```
switch (day) {
    case 1 -> System.out.println("One");
    case 2 -> System.out.println("Two");
    case 3 -> System.out.println("Three");
    default -> System.out.println("Other");
}
```

Java 14+: only the matching rule runs. No break is needed.

Java: ArrowSwitch.java

A switch expression produces a value

```
String label = switch (day) {
    case 1 -> "One";
    case 2 -> "Two";
    case 3 -> "Three";
    default -> "Other";
};
System.out.println(label);
```

Java 14+: this switch initializes label and ends with a semicolon.

Java: SwitchExpression.java

Comma-separated cases share one result

```
String category = switch (day) {
    case 1, 2, 3, 4, 5 -> "Weekday";
    case 0, 6 -> "Weekend";
    default -> "Invalid day";
};
System.out.println(category);
```

Java 14+: list the matching constants before one arrow.

Java: CombinedCases.java

yield supplies a value from a rule block

```
String result = switch (month) {
    case 2 -> {
        if (leap) yield "29 days";
        else yield "28 days";
    }
    case 4, 6, 9, 11 -> "30 days";
    case 1, 3, 5, 7, 8, 10, 12 -> "31 days";
    default -> "Invalid month";
};
System.out.println(result);
```

Java 14+: yield exits the switch expression with a result.

Java: YieldMonthDays.java

Quiz 6: switch control flow

1. What does break leave in these examples?
2. Can colon cases execute later case statements?
3. Can arrow rules fall through to another rule?
4. What must a switch expression produce?
5. Which keyword supplies a value from
   a block inside a switch expression?

The conditional operator chooses a value

```
int x = 0;
int y = (x > 0) ? 1 : -1;
System.out.println(y);
int number = 8;
String kind = (number % 2 == 0) ? "even" : "odd";
System.out.println(number + " is " + kind);
```

condition ? valueIfTrue : valueIfFalse

Java: ConditionalOperator.java

Only the selected value expression runs

```
Scanner input = new Scanner(System.in);
System.out.print("Enter number and divisor: ");
int number = input.nextInt();
int divisor = input.nextInt();
String result = divisor == 0
    ? "No quotient"
    : "Quotient: " + (number / divisor);
System.out.println(result);
```

The division is skipped when divisor equals zero.

Java: ConditionalGuard.java

Precedence among operators used here

Higher to lower	Examples
Arithmetic	*, /, % before +, -
Comparison and equality	<, <=, >, >= before ==, !=
Boolean combination	^ before && before ||
Value choice and assignment	?: before =, +=, and related assignments

Unary ! binds tightly. Parentheses make a condition easier to read.

Group arithmetic, comparisons, then logic

```
int x = 10, y = 3;
System.out.println(3 + 5 * (4 - 3) > 34);
System.out.println(x > 5 || y < 0 && x < 0);
System.out.println((x > 5 || y < 0) && x < 0);
System.out.println(!false && false || true);
```

The first line evaluates 8 > 34, which is false.

Java: ConditionPrecedence.java

Associativity determines grouping

```
System.out.println(20 - 5 - 3);
System.out.println(20 - (5 - 3));
int a, b;
a = b = 7;
System.out.println(a + " " + b);
boolean result = true ? false : true ? true : false;
System.out.println(result);
```

Subtraction groups left to right. Assignment groups right to left.

Java: AssociativityDemo.java

Quiz 7: grouping and evaluation

1. 3 + 5 * (4 - 3) > 34
2. true || false && false
3. (true || false) && false
4. 20 - 5 - 3
5. For a = b = 7, which assignment happens first?

Java: PrecedenceQuiz.java

A practical debugging routine

State the expected result for one small input.
Trace each condition and the branch it selects.
Inspect values at the first unexpected decision.
Repair one cause and rerun nearby test cases.

A debugger can pause execution and show variables.

Use a trace to expose a boundary bug

```
int score = 80;
System.out.println("DEBUG score=" + score);
System.out.println("DEBUG score>80=" + (score > 80));
if (score > 80) {
    System.out.println("At least 80");
} else {
    System.out.println("Below 80");
}
```

The intended message says at least 80, but the condition excludes 80.

Java: DebugGradeTrace.java

Putting selections together: a calculator

```
switch (operator) {
    case "+" -> System.out.println(left + right);
    case "-" -> System.out.println(left - right);
    case "*" -> System.out.println(left * right);
    case "/" -> {
        if (right == 0) System.out.println("Zero divisor");
        else System.out.println(left / right);
    }
    default -> System.out.println("Unknown operator");
}
```

Java 14+: switch chooses the operation; if guards division.

Java: Calculator.java

Test decisions, boundaries, and guards

Example	Useful tests
Grades	79.9, 80, 80.1; -1 and 101
Tiered pay	40, 41, 54, 55 hours
Leap years	2023, 2024, 1900, 2000
Calculator	5 / 2, 5 / 0, unknown operator

Every test should have an expected path and result.

Exit exercise: classify a triangle

Read three positive integer side lengths.
Restrict each side to at most 1,000,000.
Reject a triple that cannot form a triangle.
Otherwise classify: equilateral, isosceles, or scalene.

Predict outcomes for 3 3 3, 3 3 4, 3 4 5, and 1 2 3.

Exit exercise: one possible solution

```
if (a + b <= c || a + c <= b || b + c <= a) {
    System.out.println("Not a triangle");
} else if (a == b && b == c) {
    System.out.println("Equilateral");
} else if (a == b || a == c || b == c) {
    System.out.println("Isosceles");
} else {
    System.out.println("Scalene");
}
```

The complete file validates the side range before checking these conditions.

Java: TriangleClassifier.java

Chapter review

Choose a structure that matches the requirement.
Trace the first matching branch and the skipped work.
Make range boundaries and exceptions explicit.
Use short-circuit guards in the right order.
Distinguish switch statements from expressions.
Test normal, boundary, and rejected inputs.